



14.7. Shell Sort

Previous Section:

 [14.6. Quick Sort](#)

Key Section Review:

 [14.2. Selection Sort & Insertion Sort](#)

(Focus on the Insertion Sort part)

Shell Sort is an improved version of **Insertion Sort** that significantly reduces the number of element shifts required during sorting. Instead of comparing only adjacent elements as Insertion Sort does, Shell Sort first compares elements that are far apart and gradually reduces the distance between them.

This distance is called the **interval** (or **gap**). By moving elements closer to their correct positions early in the algorithm, the final pass with an interval of 1 (which is simply Insertion Sort) requires much less work.

Compared with Selection Sort and Insertion Sort, Shell Sort is much more efficient for medium-sized datasets because it reduces the number of long-distance movements needed to place elements in their correct positions.

The time complexity of Shell Sort depends on the chosen gap sequence. Using the original Shell sequence, the worst-case time complexity is approximately $O(n^2)$, while the space complexity is $O(1)$ since the algorithm sorts the array in place.

1. The Shell Sort Algorithm

The main idea behind Shell Sort is to repeatedly perform **Insertion Sort** on elements that are a fixed distance apart.

Suppose the interval (gap) is **4**. For an array:

```
Index : 0  1  2  3  4  5  6  7
Value : 35 33 42 10 14 19 27 44
```

Each element is compared with another element that is **4 positions away**.

- Index Pair: $0 \leftrightarrow 4$; $1 \leftrightarrow 5$; $2 \leftrightarrow 6$; $3 \leftrightarrow 7$
- Which forms the following sub-arrays: **{35, 14}**; **{33, 19}**; **{42, 27}**; **{10, 44}**

Each sub-array is sorted using **Insertion Sort**.

After completing all sub-arrays for the current interval, the interval is reduced.

A common interval sequence is the **original Shell sequence**: $n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

Eventually, when the interval becomes **1**, the algorithm performs a normal Insertion Sort. Since most elements are already close to their correct positions, this final pass is very efficient.

2. Demonstration of Shell Sort

Consider the following array:

```
Initial Array

[35] [33] [42] [10] [14] [19] [27] [44]
```

The array size is **8**, so the initial interval is $\text{Gap} = 8 / 2 = 4$

- Pass 1 — Gap = 4

The sub-arrays are **{35, 14}**; **{33, 19}**; **{42, 27}**; **{10, 44}**.

Sorting each sub-array gives **{14, 35}**; **{19, 33}**; **{27, 42}**; **{10, 44}**

Updating the original array:

```
Before

[35] [33] [42] [10] [14] [19] [27] [44]
```

After Gap = 4

[14] [19] [27] [10] [35] [33] [42] [44]

- Pass 2 — Gap = 2: Reduce the interval → $\text{Gap} = 4 / 2 = 2$

Now the array is divided into two larger sub-arrays.

Even indices: {14, 27, 35, 42}

Odd indices: {19, 10, 33, 44}

Apply Insertion Sort to each sub-array. The first sub-array is already sorted. The second sub-array becomes {10, 19, 33, 44}

Updating the original array:

Before

[14] [19] [27] [10] [35] [33] [42] [44]

After Gap = 2

[14] [10] [27] [19] [35] [33] [42] [44]

- Pass 3 — Gap = 1: Reduce the interval again → $\text{Gap} = 2 / 2 = 1$

Now every adjacent element belongs to the same sub-array: {14, 10, 27, 19, 35, 33, 42, 44}. This is simply an ordinary Insertion Sort.

Performing the final insertion sort gives

Before

[14] [10] [27] [19] [35] [33] [42] [44]

After Gap = 1

[10] [14] [19] [27] [33] [35] [42] [44]

The array is now completely sorted.

3. Implementation of Shell Sort

Shell Sort is implemented by repeatedly reducing the interval (gap) between compared elements.

```
def shell_sort(a): # [Time Complexity:  $O(n^2)$ ; Space Complexity:  $O(1)$ ]
    n = len(a)
    # Rearrange elements at each  $n/2, n/4, n/8, \dots, 1$  interval
    interval = n // 2 # Initialize the interval by length // 2
    while interval > 0:
        for i in range(interval, n):
            temp = a[i]
            j = i
            while j >= interval and a[j - interval] > temp:
                # Assign a[j] to a[j - interval]
                a[j] = a[j - interval]
                j -= interval
            a[j] = temp
        interval = interval // 2 # Reduce the interval
    return a
```

For each interval:

- The array is divided into several interleaved sub-arrays.
- Each sub-array is sorted using **Insertion Sort**.
- The interval is reduced according to the selected gap sequence.
- The process continues until the interval becomes **1**, where a final Insertion Sort completes the sorting.

Different gap sequences can be used to improve performance. Although Shell's original sequence ($n/2, n/4, \dots, 1$) is the simplest and most widely taught, more advanced sequences such as **Hibbard**, **Knuth**, **Sedgewick**, and **Tokuda** generally provide better practical performance.

Summary

Shell Sort is an optimization of Insertion Sort that first sorts elements far apart before gradually reducing the interval between compared elements.

By moving elements closer to their correct positions early in the sorting process, the final pass with an interval of **1** requires significantly fewer shifts than a standard Insertion Sort.

Although its worst-case time complexity is approximately $O(n^2)$ when using the original Shell gap sequence, Shell Sort typically performs much faster in practice while requiring only $O(1)$ additional memory.

It is an efficient in-place sorting algorithm for medium-sized datasets and serves as an important improvement over the basic quadratic sorting algorithms.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive_link

<https://www.programiz.com/dsa/shell-sort>

<https://www.geeksforgeeks.org/dsa/shell-sort/>

Next Section:

 [14.8. Cocktail Sort & Odd-Even Sort](#)